



Cairo University
Faculty of Economics and Political Science
Socio-Computing Department
C201: Computer Applications in Social Sciences II

MODULE 2: PYTHON FUNDAMENTALS

LECTURE 2_4_PART1

PYTHON COLLECTION DATA TYPES:

LISTS

DR. AMAL SAKR

OUTLINES

Python Fundamentals

Previous Lectures: Variables, Operators, and Expressions, Condition (if...Else), Loops (For and While Statements), Functions and Modules

This Lecture: *Python Collection Data Types- Lists*

- What are Python Collection Data Types?
- Lists: Definition and Accessing their elements
- List Slicing
- List Methods and Built-in Functions
- Looping Over a List
- List Comprehension
- List of Strings: String methods, Looping Over a String and a List of Strings, and List Comprehension using String

Python Collection Data Types

Python provides several collection data types that allow users to store and manage groups of data efficiently.

The four main built-in collection types are lists, tuples, sets, and dictionaries.

1. A **list** is an **ordered, mutable** collection that can store **different data types**, making it highly flexible.
2. A **tuple** is **similar to a list** but is **immutable**, meaning its elements cannot be changed after creation, which makes it useful for storing fixed data.
3. A **set** is an **unordered** collection of **unique** elements, often used for mathematical operations like union and intersection (*out of concern*).
4. Finally, a **dictionary** is a collection of **key-value pairs**, enabling fast data retrieval by keys rather than index positions.

List	[1, 2, 3, 4, 5, 6]
Tuple	(1, 2, 3, 4, 5)
Set	{1, 4, 14, 33} → <i>not studied in our course</i>
Dictionary	{"Name": "Anita", "Age": 77, "job": "Teacher"}

For details about python data structures refer to

<https://docs.python.org/3/tutorial/datastructures.html#more-on-lists>

List

- A list is a sequence of items of *same* or *different* of *data types* (floats, integers, strings or even other lists) which is **ordered** and **modifiable(mutable)**.
- Lists are essential to Python programming and are used to store collections of other values.
- `[ ]` is used to contain the indices of the item in a list, where the **first index** is **0**, and the **last index** is **n-1** (assuming the list has n items)

```
x =[]      #An empty list
type(x)    #Out: list
x =[1,2,3,4] # 1-dimensional list
y = [[1,2,3,4], [5,6,7,8]] # 2-dimensional list (list of lists)
z = [1,1.0,'one',False] # Mixed List
```

List: *Slicing*

- List slicing in Python allows to extract a portion of a list using a special syntax. It helps in retrieving subsets of a list without modifying the original data.
- Slicing uses `[]` to contain the indices of the items in a list. Remember that the first index in the list is 0, and the last is $(n - 1)$ (assuming the list has n items)
- **Basic Slicing Syntax:** `list[start:stop:step]`

start → The index where slicing begins (inclusive); (defaults = 0).

stop → The index where slicing ends (exclusive); (defaults = `len(list)`)

step → The number of steps to skip (optional); (defaults = 1).

Example: Assume the list `nums = [10, 20, 30, 40, 50, 60, 70]`

Slice	Result	Description
<code>nums[1:4]</code>	<code>[20, 30, 40]</code>	Elements from index 1 to 3
<code>nums[:3]</code>	<code>[10, 20, 30]</code>	Elements from index 0 to 2
<code>nums[2:]</code>	<code>[30, 40, 50, 60, 70]</code>	From index 2 to end
<code>nums[::2]</code>	<code>[10, 30, 50, 70]</code>	Every 2 elements

List: *Slicing*

Slicing with Negative Indices

When slicing lists in Python, you can use negative indices as well. This allows you to reference elements from the end of the list.

For example, the index -1 represents the last element ($n - 1$), -2 represents the second-to-last element ($n - 2$), and so on.

Example: Assume the list **nums** = [10, 20, 30, 40, 50, 60, 70]

Slice	Result	Description
nums[-3:]	[50, 60, 70]	Last 3 elements
nums[: : -1]	[70, 60, 50, 40, 30, 20, 10]	Reverse the list

nums elements	10	20	30	40	50	60	70
+ index	0	1	2	3	4	5	6
- index	-7	-6	-5	-4	-3	-2	-1

List: *Slicing*

Example

```
x = [0,10,20,30,40,50,60,70,80,90]
# accessing list items using positive and negative indexing
x[0], x[5], x[9], x[-1], x[-6]      #Out: (0, 50, 90, 90, 40)
x[10]                                #Out: list index out of range

x[:4]                                #Out: [0, 10, 20, 30]
x[1:4]                              #Out: [10, 20, 30]
x[1:4:2]                            #Out: [10, 30]

y = [[10,20,30,40], [50,60,70,80]] # list of lists
y[0]                                #Out: [10,20,30,40],
y[1]                                #Out: [50,60,70,80]
y[0][0]                             #Out: 10
y[0][1:4]                           #Out: [20, 30, 40]
y[1][1:4]                           #Out: [60, 70, 80]

z=[x, [20,70]];z                    #Out: [[0, 10, 20, 30, 40, 50, 60, 70, 80,
90], [20, 70]]
```

List Methods

A method is a function associated with a list. It is called using dot notation (`object.method()`).

Python has a set of built-in methods that you can use on lists. The most useful are: `append`, `extend`, `pop`, `remove` and `count`.

Function	Description
<code>append(x)</code>	Adds x to the end
<code>insert(i, x)</code>	Inserts x at index i
<code>extend(sequence)</code>	Adds multiple elements
<code>remove(x, value)</code>	Removes value from x
<code>pop(i)</code>	Removes element at index i and returns it.
<code>clear()</code>	Removes all elements
<code>sort()</code>	Sorts the list in ascending order
<code>sort(reverse=True)</code>	Sorts in descending order
<code>reverse()</code>	Reverses the list
<code>index(x)</code>	Returns index of x
<code>count(x)</code>	Counts occurrences of x
<code>copy()</code>	Copies the list

For example,

```
x = [2, 6, 9, 15]
x.append(7); print(x) #out [2, 6, 9, 15, 7]
```

Built-in Functions Applied to Lists

Python has a set of built-in functions that you can use on lists. The most useful are *min*, *max*, *sorted*, *reversed*, and *dir*.

Function	Description
<code>dir(list)</code>	Returns a list of valid attributes and methods for an object.
<code>len(list)</code>	Returns the number of items in the list.
<code>sum(list)</code>	Returns the sum of all numeric elements in the list.
<code>max(list)</code>	Returns the largest item in the list.
<code>min(list)</code>	Returns the smallest item in the list.
<code>sorted(list)</code>	Returns a new sorted list (doesn't modify the original).
<code>reversed(list)</code>	Returns an iterator that accesses the list in reverse order.
<code>any(list)</code>	Returns True if at least one element is truthy.
<code>all(list)</code>	Returns True if all elements are truthy.
<code>print(list)</code>	Outputs the list to the console.

In Python, methods and built-in functions operate on objects like lists, but their syntax differs slightly. *For example,*

```
x = [2, 6, 9, 15]
L = len(x); print(L) #out 4
```

dir() Function

The `dir()` function returns all properties and methods of the specified object, without the values. This function will return all the properties and methods, even built-in properties which are default for all object.

Syntax: `dir(object)`

Int, *string*, *list* are an example of Python objects. An object contains both

- **data** (the actual string itself) and
- **methods**, which are effectively functions that are built into the object and are available to any instance of the object.

```
x = [0,10,20,30,40,50]
```

```
y = 'High'
```

```
dir(x) # retrieve all properties and methods of list Type
```

```
dir(y) # retrieve all properties and methods of string Type
```

List Methods

```
y = [0, 10, 20, 30, 40, 50, 60, 70, 80, 90]
x = y; len(x)      #Out: 10
x.append(100);      # add 100 to the list
print(x)           #Out: x = [0,10,20,30,40,50,60,70,80,90,100]
len(x)             #Out: 11
print(y)
# Out: [0,10,20,30,40,50,60,70,80,90,100]
```

Note

`x = y` does not create a new list — it simply makes x and y refer to the same object in memory.

So, when you print y, you see the modified list — because y and x are just two names for the same list.

How to prevent this?

Create an independent copy: `x = y.copy()`

Now x and y are different objects, so modifying x won't affect y.

List Methods

```
y = [0, 10, 20, 30, 40, 50, 60, 70, 80, 90]
x = y.copy(); len(x) #Out: 10
x.append(100)
print(len(x), x) #Out: 11 [0,10,20,30,40,50,60,70,80,90, 100]
x.append([11,12,13]);x
#Out: [0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100, [11, 12, 13]]
print(len(x), x[0]) #Out: 12 0

x = y.copy() # Return to the original list
x = [x, [11,12,13]] ; x # Create list of lists
# Out:[0, 10, 20, 30, 40, 50, 60, 70, 80, 90], [11, 12, 13]]
print(len(x), x[0]) #Out: 2 [0, 10, 20, 30, 40, 50, 60, 70, 80, 90]

x = y.copy()
x.extend([11,12,13]); x
#Out: [0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 11, 12, 13]
print(len(x), x[0]) #Out: 13 0

x = x + [14,15,16] ;x
#Out:[0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 11, 12, 13, 14, 15, 16]
```

List Methods

```
y = [0, 10, 20, 30, 40, 50, 60, 70, 80, 90]
```

```
w = ['first', 'second', 'third']
```

```
x = y.copy(); x.append(w); print(len(x),x)
```

```
#Out: 11 [0, 10, 20, 30, 40, 50, 60, 70, 80, 90,  
['first', 'second', 'third']]
```

```
x = y.copy(); x.extend(w) ; print(len(x),x)
```

```
#Out: 13 [0, 10, 20, 30, 40, 50, 60, 70, 80, 90,  
'first', 'second', 'third']
```

```
x = y.copy();
```

```
x = x + w ; print(len(x),x) #Same result as extend
```

```
#Out: 13 [0, 10, 20, 30, 40, 50, 60, 70, 80, 90,  
'first', 'second', 'third']
```

```
# Note: *, -, / are not supported operands between lists
```

```
x = [y,w] ; print(len(x),x) #Create list of lists
```

```
#Out:2 [[0, 10, 20, 30, 40, 50, 60, 70, 80, 90],  
['first', 'second', 'third']]
```

List Methods

```
y = [0, 10, 20, 30, 40, 50, 60, 70, 80, 90]
```

```
w = ['first', 'second', 'third']
```

```
x = y.copy()
```

```
del x[1:3]; x          #Out: [0, 30, 40, 50, 60, 70, 80, 90]
```

```
print(x.pop(1), x) #Out: 30 [0, 40, 50, 60, 70, 80, 90]
```

#Out: pop same effect as del but return the deleted item

```
del x[0]; x           #Out:[40, 50, 60, 70, 80, 90]
```

#delete using index

```
x.remove(80); x       #Out: [40, 50, 60, 70, 90]
```

delete specific value

```
x.clear(); x         #Out: []
```

```
del x[:]; x          #Out: [] Same as clear
```

List Methods

```
y = [1, 8, 7, 2, 3, 4, 4, 5, 6, 9, 9]
```

```
y.count(1) , y.count(4)          #Out: (1, 2)
```

```
y.reverse(); y                  #Out: [9, 9, 6, 5, 4, 4, 3, 2, 7, 8, 1]
```

```
y.sort(); y                     #Out: [1, 2, 3, 4, 4, 5, 6, 7, 8, 9, 9]
```

```
y.sort(reverse = True); y  
                                #Out: [9, 9, 8, 7, 6, 5, 4, 4, 3, 2, 1]
```

```
does_exist = 20 in y  
            #Checking if an items exists in a List
```

```
print(does_exist)               #Out: False
```

```
y.clear()  
            #The clear()method empties the list
```

Built-in Functions

Note the difference between methods and built-in functions

```
x = [35, 20, 30, 15]
```

```
print(max(x), min(x), sum(x))    #Out: (35, 15, 100)
```

```
print(sorted(x))
```

#sorted is a built-in function Out: [15, 20, 30, 35]

```
print(x)    #Out: [35, 20, 30, 15] → x has not changed
```

```
x.sort()    #sort is a list method
```

```
print(x)    #Out: [15, 20, 30, 35] → x has changed
```

```
st_code = [1023, 1011, 1045, 1032]    # student code
```

```
st_name = ["Mona", "Omar", "Sara", "Ali"]
```

```
x = zip(st_code, st_name)
```

```
x= list(x); print("Before sorting:", x)
```

```
x.sort(); print("ascending sorting:", x)
```

```
x.sort(reverse=True); print("descending sorting:", x)
```

zip is a built-in Python function that performs an element-wise combination of sequences. It works with two or more iterable objects.

Example: Calculating Median

Write a function to find the median of a **list** of income values.

◆ *Example Input:* [2500, 3200, 4000, 2800, 5000]

◆ *Expected Output:* Median income: \$3200

```
def median_income(incomes):  
    sorted_incomes = sorted(incomes)  
    n = len(sorted_incomes)  
    mid = n // 2  
    if n % 2 == 0:  
        return (sorted_incomes[mid] + sorted_incomes[mid-1]) / 2  
    return sorted_incomes[mid]  
  
incomes = [2500, 3200, 4000, 2800, 5000]  
mid_value = median_income(incomes)  
print("Median income: ", round(mid_value,2))
```

Example: Calculating GDP

Given a **list** of GDP values over years, calculate the growth rate for each year.

- ◆ Example Input: [2.1, 2.3, 2.6, 2.9] (trillion dollars)
- ◆ Expected Output: Growth Rates: [9.52%, 13.04%, 11.54%]

GDP Growth Rate

$$= \frac{\text{GDP in current year} - \text{GDP in previous year}}{\text{GDP in previous year}} \times 100$$

Example: Calculating GDP

$$\text{GDP Growth Rate} = \frac{\text{GDP in current year} - \text{GDP in previous year}}{\text{GDP in previous year}} \times 100$$

```
def gdp_growth_rate(gdp_values):  
    growth_rates = []  
    N = len(gdp_values)  
    for i in range(1,N):  
        rate = (gdp_values[i] - gdp_values[i - 1]) /  
        gdp_values[i - 1] * 100  
        rate = str(round(rate,2))+"%"  
        growth_rates.append(rate)  
    return growth_rates
```

```
gdp_values = [2.1, 2.3, 2.6, 2.9] #trillion dollars  
result = gdp_growth_rate(gdp_values)  
print("Growth Rates:", result)  
# Out: Growth Rates: ['9.52%', '13.04%', '11.54%']
```

Looping Over a List: *List Comprehension*

List comprehension in Python is a compact way of creating a list from a sequence. It is a short way to create a new list. List comprehension is considerably faster than processing a list using standard `for` loop.

The Syntax

```
newlist = [expression for item in sequence if condition == True]
```

- ***expression***: Represents the operation you want to execute on every item within the sequence.
- ***item***: refers to each value taken from the sequence.
- ***sequence***: specify the sequence of elements you want to iterate through.(e.g., a list, tuple, or string).
- ***Condition (Optional)***: A filter helps decide whether or not an element should be added to the new list.

```
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]  
copy_list = [x for x in fruits]  
print(copy_list)  
new_list = [x for x in fruits if x != "apple"]  
print(new_list)
```

Looping Over a List: *List Comprehension*

```
n=100; even_numbers=[]  
  
# --- Generating even numbers within a range using regular For St.  
for i in range(n+1): if i%2==0: even_numbers.append(i)  
  
# Generating even numbers within a range using List Comprehension  
even_numbers = [i for i in range(n+1) if i % 2 == 0]  
    # to generate even numbers list in range 0 to 20  
print(even_numbers)    # [0, 2, 4, 6, 8, 10, 12, ...]  
  
# ----- Generating odd numbers within a range  
odd_numbers = [i for i in range(1:n) if i % 2 != 0]  
    # to generate odd numbers in range 0 to 20  
print(odd_numbers)    # [1, 3, 5, 7, 9, 11, 13,...]  
  
# ----- Filter out positive even numbers from a list  
numbers = [-8, -7, -3, -1, 0, 1, 3, 4, 5, 7, 6, 8, 10]  
pos_even_num = [i for i in numbers if i % 2 == 0 and i > 0]  
print(pos_even_num)    # [4, 6, 8, 10]
```

String Methods

Python treats anything inside quotes as a string. This includes **letters**, **numbers**, and **symbols**. Python has a set of built-in methods that you can use on strings. Below are some examples for these methods.

For more information about string methods refer to

https://www.w3schools.com/python/python_ref_string.asp

Function	Description
<u>capitalize()</u>	Converts the first character to upper case
<u>isdigit()</u>	Returns True if all characters in the string are digits
<u>count()</u>	Returns the number of times a specified value occurs in a string
<u>find()</u>	Searches the string for a specified value and returns the position of where it was found
<u>index()</u>	Searches the string for a specified value and returns the position of where it was found
<u>join()</u>	Converts the elements of an iterable into a string
<u>lower()</u>	Converts a string into lower case
<u>replace()</u>	Returns a string where a specified value is replaced with a specified value
<u>split()</u>	Splits the string at the specified separator, and returns a list
<u>upper()</u>	Converts a string into upper case
<u>strip()</u>	Remove spaces and any user defined characters from a string

Example

```
txt_news = "BBC: babies have died from cold in Gaza over  
two weeks"  
  
print (txt_news.upper())    #out: BBC: BABIES HAVE ...  
print (txt_news.lower())    #out: bbc: babies have ...  
L = len(txt_news)  
  
print("No. of characters = ",L)    #Out: L = 54  
  
words= txt_news.split()  
print("Words in text news = ",words)  
    # Out: ['BBC:', 'babies', 'have', ...  
  
print("No. of words = ",len(words))  
    # Out: Words in text news= 11  
  
sent= txt_news.split("Gaza"); print(sent)  
# Out: ['BBC: babies have died from cold in ', 'over two weeks']  
  
print("No of Sentences = ", len(sent))  
    # Out: No of Sentences = 2
```

Example

```
txt_news = "BBC: babies have died from cold in Gaza over  
two weeks"
```

```
Target = "Palestine"
```

```
x = txt_news.find(Target)
```

```
#if found it returns the position of the word Palestine
```

```
print (x) #out: -1 because it is not existed in the text
```

```
Target = "Gaza";
```

```
x=txt_news.find(Target); print (x) #out: 35
```

```
x=txt_news.count(Target)
```

```
#returns count of word Gaza in the text
```

```
print(f"Count of word {Target} in text news = {x}")
```

```
d=round(x/len(words)*100,2)
```

```
print (f "Density of word: {Target} in the text = {d}%")
```

```
# remember Ex 3 in the Function lecture
```

Example

```
# Six methods to find the longest word within a sentence
```

```
def longest_word(words):
```

```
    longest_word = ""
```

```
    for w in words:
```

```
        if len(w)>len(longest_word): longest_word = w
```

```
    return longest_word
```

```
txt_news = "BBC: babies have died from cold in Gaza over  
two weeks"
```

```
words= txt_news.split()
```

```
print("Words in text news = ",words)
```

```
    # Out: ['BBC:', 'babies', 'have', ...
```

```
Lword  = longest_word(words)
```

```
print("1. User Defined Function - Longest word: ", Lword)
```

```
    # Out: babies
```

Example

```
Lword = max(words, key = len)
```

```
print("2. Max Method - Longest word: ", Lword)
```

```
x= sorted(words, key = len)
```

```
print("3. Built-in Sorted - Longest word: ", x[-1])
```

```
x= sorted(words, key = len, reverse = True)
```

```
print("4. Built-in Reverse Sorted - Longest word: ",  
x[0])
```

```
words.sort(key = len)
```

```
print("5. Sort Method - Longest word is: ", words[-1])
```

```
words.sort(key = len, reverse = True)
```

```
print("6. Reverse Sort Method - Longest word: ",  
words[0])
```

A String Slicing and Looping: *Example*

```
txt_news = "BBC: babies have died from cold in Gaza over  
two weeks"
```

```
L = len(txt_news)           #Out: L = 54
```

```
txt_news[0], txt_news[10], txt_news[L-1] #String Slicing
```

```
#Out: ('B', 's', 's')
```

```
txt_news[:10], txt_news[44:]
```

```
#Out: ('BBC: babie', ' two weeks')
```

#Loop over a string using 3 different methods

#Out for below three methods are the same B B C : b a b i.....

```
for x in txt_news: print(x) #method 1
```

```
for i in range(L): print(txt_news[i]) #method 2
```

```
i=0 #method 3
```

```
while i<L:
```

```
    print(txt_news[i]);    i+=1
```

List of Strings

```
first_names = ["Nader", "Noha", "Mostfa", "Alia"]
last_names = ["Ahmed", "Ibrahim", "Kamal", "Ameer"]
names = [first_names, last_names]; print(names)
#Out: [['Nader', 'Noha', 'Mostfa', 'Alia'],
#      ['Ahmed', 'Ibrahim', 'Kamal', 'Ameer']]

# Accessing elements within a list of strings
print(names[0]) #Out: ['Nader', 'Noha', 'Mostfa', 'Alia']
print(names[1]) #Out: ["Ahmed", "Ibrahim", "Kamal", "Ameer"]
print(names[0][1], names[1][0], names[1][2:3], names[0][0:2])
#Out: Noha, Ahmed, ['Kamal'], ['Nader', 'Noha']

students = [ ["Ahmed", "Ali"], ["Dina", "Khaled"], ["Lina", "Ameer"] ]
print(students[0])      #Out: ['Ahmed', 'Ali']
print(students[1])      #Out: ['Dina', 'Khaled']
print(students[2][0])   #Out: Lina
print(students[2][1])   #Out: Ameer
```

Example: Extract Email Providers (Domains)

Write a Python function that takes a list of email addresses and returns a list of unique email providers (*domains are the text after the @ symbol*).

```
def email_provider(emails):
    domains = [] # List to store extracted domains
    for email in emails:
        at_index = email.find("@") # Find the index of '@'
        if at_index == -1: continue # if not exist go next
        ES = email.split('@')
        domain = ES[1]
        if domain not in domains: # Avoid duplicates
            domains.append(domain)
    return sorted(domains) # Return sorted unique domains

emails = [ "alice@gmail.com", "bob@yahoo.com",
           "charlie@gmail.com", "david@outlook.com",
           "eve@yahoo.com", "frank@company.com" ]
print(email_provider(emails))
```

Looping Over a List of Strings

List Comprehension

```
#Loop over a list of strings -- you can similarly use  
while st.
```

```
first_names = ["Nader", "Noha", "Mostfa", "Alia"]
```

```
last_names = ["Ahmed", "Ibrahim", "Kamal", "Ameer"]
```

```
full_name = []
```

```
L = len(first_names)
```

```
for i in range(L):
```

```
    N = first_names[i]+" "+last_names[i]
```

```
    full_name.append(N)
```

```
print(full_name)
```

```
#Out: ['Nader Ahmed', 'Noha Ibrahim', 'Mostfa Kamal',  
'Alia Ameer']
```

Looping Over a List of Strings

List Comprehension

```
#----- Loop to create a list of full names
```

```
students = [["Ahmed", "Ali"], ["Dina", "Khaled"],  
["Lina", "Ameer"]]
```

```
full_name=[]
```

```
for s in students: full_name.append(s[0]+" "+s[1])
```

```
print(full_name)
```

```
#Out: ['Ahmed Ali', 'Dina Khaled', 'Lina Ameer']
```

```
#- Create a list of full names using List Comprehension
```

```
full_name = [s[0] + " " + s[1] for s in students]
```

```
print(full_name)
```

```
#Out: ['Ahmed Ali', 'Dina Khaled', 'Lina Ameer']
```

Sheet 4: Ex 5

Define a function named `Gini_Ind(incomes)` that takes a list of income values as input and calculates their Gini index (GI), a common measure of income inequality within a country or population. It ranges from 0 to 1.

Gini Coefficient is calculated as follows

$$GI = \frac{\sum_{i=1}^n \sum_{j=1}^n |x_i - x_j|}{2n^2 \bar{x}}$$

For a sorted data it can be calculated using the simplified formula:

$$GI = \frac{\sum_{i=1}^n (2i - n - 1)x_i}{n \sum_{i=1}^n x_i}$$

Where:

- x_i, x_j are income values of individuals
- n is the total population, and $\bar{x}$ is the mean income

The values of GI are interpreted as follows

- $0 \leq GI \leq .3$ refers to low inequality (e.g., Scandinavian countries)
- $.3 < GI \leq .5$ refers to moderate inequality (e.g., USA, UK)
- $.5 < GI \leq 1$ refers to high inequality (e.g., some developing countries)

Answer

$$GI = \frac{\sum_{i=1}^n (2i-n-1)x_i}{n \sum_{i=1}^n x_i}$$

```
def gini_coefficient(data):  
    sorted_data = sorted(data) # Sort data in ascending order  
    n = len(sorted_data); total = sum(sorted_data)  
    numerator = 0; rank = 1 # rank goes from 1 to n  
    for income in sorted_data:  
        contribution = income * (2 * rank - (n + 1))  
        numerator += contribution  
        rank += 1 # Increment rank  
    G = round(numerator / (n * total), 2) # Compute Gini  
    if 0 <= G <= 0.3: interpretation = "Low "  
    elif 0.3 < G <= 0.5: interpretation = "Moderate"  
    elif 0.5 < G <= 1: interpretation = "High"  
    else: interpretation = "Invalid Gini Index"  
    return G, interpretation  
  
incomes = [500, 2000, 1500, 3000, 1000] # unsorted  
gini, interp = gini_coefficient(incomes)  
print(f"Gini coefficient: {gini}, {interp} Inequality ")
```

Sheet 4: Ex 11

Refer to Exercise 9 in Sheet 3 and update the function to accept a user-input list.

Convert all elements to lowercase to handle inconsistent capitalization, and replace the loop used for counting elements with the list's `count()` method.

Remember the answer in the previous lecture

```
def analyze_list(data_list, word1, word2):  
    count1 = 0;      count2 = 0;      total = len(data_list)  
    for item in data_list: # Loop through the list  
        if item == word1: count1 += 1  
        elif item == word2: count2 += 1  
  
        # Display results  
    percent1 = (count1 / total) * 100  
    percent2 = (count2 / total) * 100  
    print(word1, ":", count1, "(", round(percent1, 2), "% )")  
    print(word2, ":", count2, "(", round(percent2, 2), "% )")  
  
    # Determine which one is higher  
    if count1 > count2: print(f"The Majority is {word1}.\n")  
    elif count2 > count1: print(f"The Majority is {word2}.\n")  
    else: print("The counts are equal\n")  
  
# Example 1: Genders list  
genders = ["Male", "Female", "Female", "Male", "Female", "Female"]  
analyze_list(genders, "Male", "Female")  
  
# Example 2: Voters' list  
votes = ["Mamdani", "Cuomo", "Mamdani", "Mamdani", "Cuomo"]  
analyze_list(votes, "Mamdani", "Cuomo")
```

Answer

```
def analyze_list(data_list, word1, word2):  
    # Convert all list items and keywords to lowercase  
    data_list = [item.lower() for item in data_list]  
    word1 = word1.lower()  
    word2 = word2.lower()  
    total = len(data_list)  
    count1 = data_list.count(word1)  
    count2 = data_list.count(word2)  
    percent1 = (count1 / total) * 100  
    percent2 = (count2 / total) * 100  
    print(f"\n{word1}: {count1} ({round(percent1, 2)}%)" )  
    print(f"{word2}: {count2} ({round(percent2, 2)}%)" )  
    if count1 > count2: print(f"The majority is {word1}.\n")  
    elif count2 > count1: print(f"The majority is {word2}.\n")  
    else: print("The counts are equal.\n")  
  
# Input list from user  
data = input("Enter list items separated by spaces: ").split()  
w1 = input("Enter first word to analyze: ")  
w2 = input("Enter second word to analyze: ")  
analyze_list(data, w1, w2) # Call the function
```

Exercise for Fun

The **Rosetta Stone** (*discovered in 1799 by François Champollion and now in the British Museum, London*) helped decode the Ancient Egyptian language (Hieroglyphic).

Write a function to simulate how Champollion cracked the Egyptian hieroglyphs.

Hint: He spotted repeated words and names, like "Ptolemy" and "king", and matched them to repeating symbols.

Function Name

```
find_repeating_matches(greek_text, hiero_text):
```

Input:

```
greek_text = ["king", "ptolemy", "beloved", "of",  
"the", "gods", "king", "ptolemy", "gave", "land",  
"to", "the", "gods"]
```

```
hiero_text = ["♀", "🦅", "❤️", "⚡", "△", "👁️",  
"♀", "🦅", "👉", "🌍", "➡️", "△", "👁️"]
```



The Rosetta Stone
Writing: *Hieroglyphs, Demotic,
and Greek scripts*

Output:

Greek Word	Glyph Symbol
king	♀
ptolemy	🦅
beloved	❤️
of	⚡
the	△
gods	👁️
gave	👉
land	🌍
to	➡️